Appendix C: IEEE 830 Template

Software Requirements Specification

for

Game ROM-Scan Analyzer

Version 5

Prepared by Kacper Pawłowski

IFE/WEEIA 4CS3

12.06.26

Table of Contents

Revision History

	Date	Reason for Changes	Version
Initial version	27.04.26	Initial draft	v1
Description and Requirements	8.05.26	Adding UML description diagram and filling out the corresponding part/.	v2
System Features	15.05.26	Adding System Features part along with a uml class diagram.	v3
Polishing	22.05.26	Updating the document and creating fresh UML diagrams	v4
Final Draft	12.06	Final draft	v5

1. Introduction

1.1 Purpose

The purpose of this document is to specify the software requirements for the ROM-Scan Analyzer, version 1.0. This product is designed to scan local directories to identify and categorize video game ROM files. The scope covers the initial development phase, focusing on the directory parsing engine and the interface for displaying ROM metadata.

1.2 Document Conventions

This specification follows the IEEE 830 standard. Bold text is used for UI elements, while italics are used for references. Every requirement statement is assigned a unique identifier (e.g., REQ-1) and a priority level: High, Medium, or Low. Higher-level requirements pass their priority down to detailed requirements.

1.3 Intended Audience and Reading Suggestions

This document is intended for Developers (to guide implementation), Laboratory Instructors (for verification), and Documentation Writers. The document is organized into six sections; it is recommended to read the Product Scope (1.4) and Overall Description (Section 2) first before proceeding to System Features.

1.4 Product Scope

Currently, digital game enthusiasts often possess thousands of ROM files spread across inconsistent directory structures, often with cryptic filenames (e.g., "smb_v1.zip"). This makes library management a manual, time-consuming process.

The ROM-Scan Analyzer addresses this problem by providing an automated parsing engine. The system will bridge the gap between "raw data" and a "user-friendly library" by identifying files based on internal headers and standard naming conventions. While the final application vision includes metadata fetching and emulator integration, this laboratory project focuses on the core challenge: the reliable identification and metadata display of local ROM files. This establishes a testable, foundational module for the larger ecosystem.

1.5 References

1. IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications.

2. Course Assignment Document: Laboratory Project Guidelines.

3. ROM Hacking/Preservation Standards: Documentation regarding common ROM file signatures.2.1. Product Perspective

2. Overall Description

2.1. Product Perspective

The ROM-Scan Analyzer System is a standalone, self-contained desktop utility. It interacts directly with the local host operating system's File System to parse directory trees and evaluate binary data blocks without modifying source data integrity.

2.2. Product Features

The core structural architecture of the system features a collection of tightly coupled modules:

- **DirectoryScanner: Autonomously traverses directory trees and isolates target extensions.**
- **RomFile: Extracts internal structural signatures for integrity validation.**
- **ScannedLibrary: Houses the runtime in-memory index of discovered titles.**
- **GameMetadata: Provides historical region and platform context.**
- **LibraryReport: Exports aggregated library details into flat files.**

2.3. User Classes and Characteristics

- **Casual Emulator User: Requires an automated, simple scanning process to index small, localized game folders.**
- **Archival Manager (High Priority): Technical users maintaining dense data structures (10,000+ items) who expect precise checksum extraction and detailed export formatting.**

2.4. Operating Environment

The utility operates natively within desktop environments (Windows, macOS, Linux). It requires read-only clearance on the host file system and runs parallel to external game emulators.

2.5. Design and Implementation Constraints

The engine is legally constrained to read-only disk access to eliminate data corruption risks. It must maintain low-level memory usage (< 512MB RAM) when parsing large nested arrays.

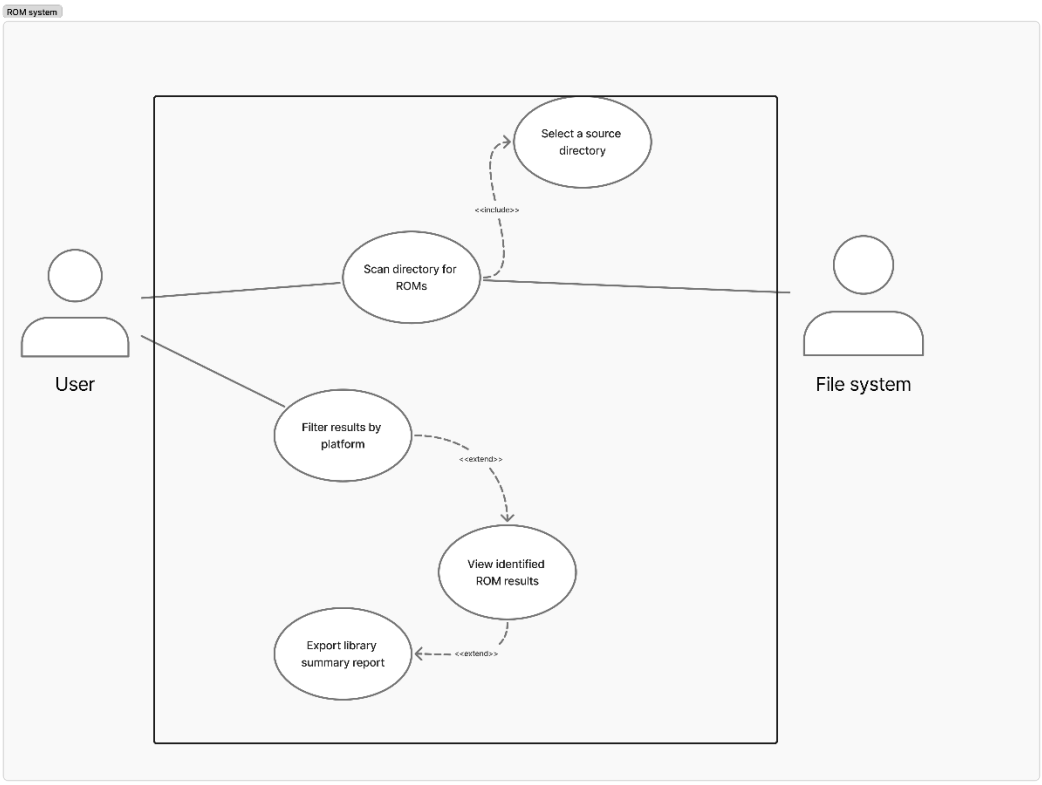
2.6. User Documentation

An installation and usage guide is delivered via a standard ReadMe.md file accompanying the application build.

2.7. Assumptions and Dependencies

The architecture assumes local ROM files adhere to recognized preservation naming conventions and standard binary header formats.

3. System Features



3.1. Feature: Select Source Directory

- **Description: The system allows the user to browse local storage media or input an explicit absolute directory path to establish the root workspace for subsequent operations.**
- **Functional Alignment: This feature serves as a mandatory pre-requisite, passing the target path variable directly to the execution loop of the scanning engine.**

3.2. Feature: Scan Directory for ROMs

- **Description: The system recursively traverses the designated file path, parses binary file structures, and validates files against an internal header signature database.**
- **Functional Alignment: This feature encapsulates the primary parsing mechanism of the utility, mapping internal file system attributes onto readable metadata.**

3.3. Feature: View Identified ROM Results

- **Description: The system displays a structured interface populated with the extracted attributes of all successfully validated ROM files.**
- **Functional Alignment: Acts as the primary data presentation layer, serving as the base screen from which users can trigger filtering or reporting sub-systems.**

3.4. Feature: Filter Results by Platform

- **Description: The system enables users to narrow the active display list dynamically using explicit criteria such as target gaming console or file type extensions.**
- **Functional Alignment: An optional, user-initiated extension that manipulates the visibility layer of the active library state without re-running a directory scan.**

3.5. Feature: Export Library Summary Report

- **Description: The system compiles the current active library configuration and writes a structured summary index to an external flat file.**

- **Functional Alignment: An optional reporting engine that outputs independent tracking metrics for historical storage use or external library verification.**

4. External Interface Requirements

4.1. User Interfaces Overview

The application utilizes a clean, single-screen **Graphical User Interface (GUI)** constructed using Python's native tkinter frame bindings (or customtkinter classes) to ensure smooth interaction.

4.1.1 Interface Navigation Workflow

1. Upon application initialization, the system loads into an idle state. The user encounters a clean, multi-row window layout.

2. The user interacts with the top panel by clicking the **Browse Folder** button. This opens a native desktop directory chooser dialog. Alternatively, the user can type or paste an absolute path directly into the **Path Entry Bar**.

3. Clicking the **Execute Scan** button dispatches a worker thread. A non-blocking status label changes to Library State: Parsing Directories....

4. Once completed, the main presentation canvas replaces its blank state with a multi-column, sorted tabular grid (ttk.Treeview) populated with row items representing parsed files.

5. The user can interact with the **Platform Filter** list to narrow rows instantaneously. Clicking **Export Report** prompts a file-save window to export the aggregated summary lo

4.2. Hardware Interfaces

The application has no direct custom physical hardware interfaces. It operates standard interface protocols via the underlying host workstation:

- **Storage Ingestion:** Interacts with magnetic disks, solid-state drives (SSDs), and external USB storage vectors through standard OS controller hardware channels.

- **User Input Devices:** Maps system interaction events via standard keyboard and pointing mouse configurations.

4.3. Software Interfaces

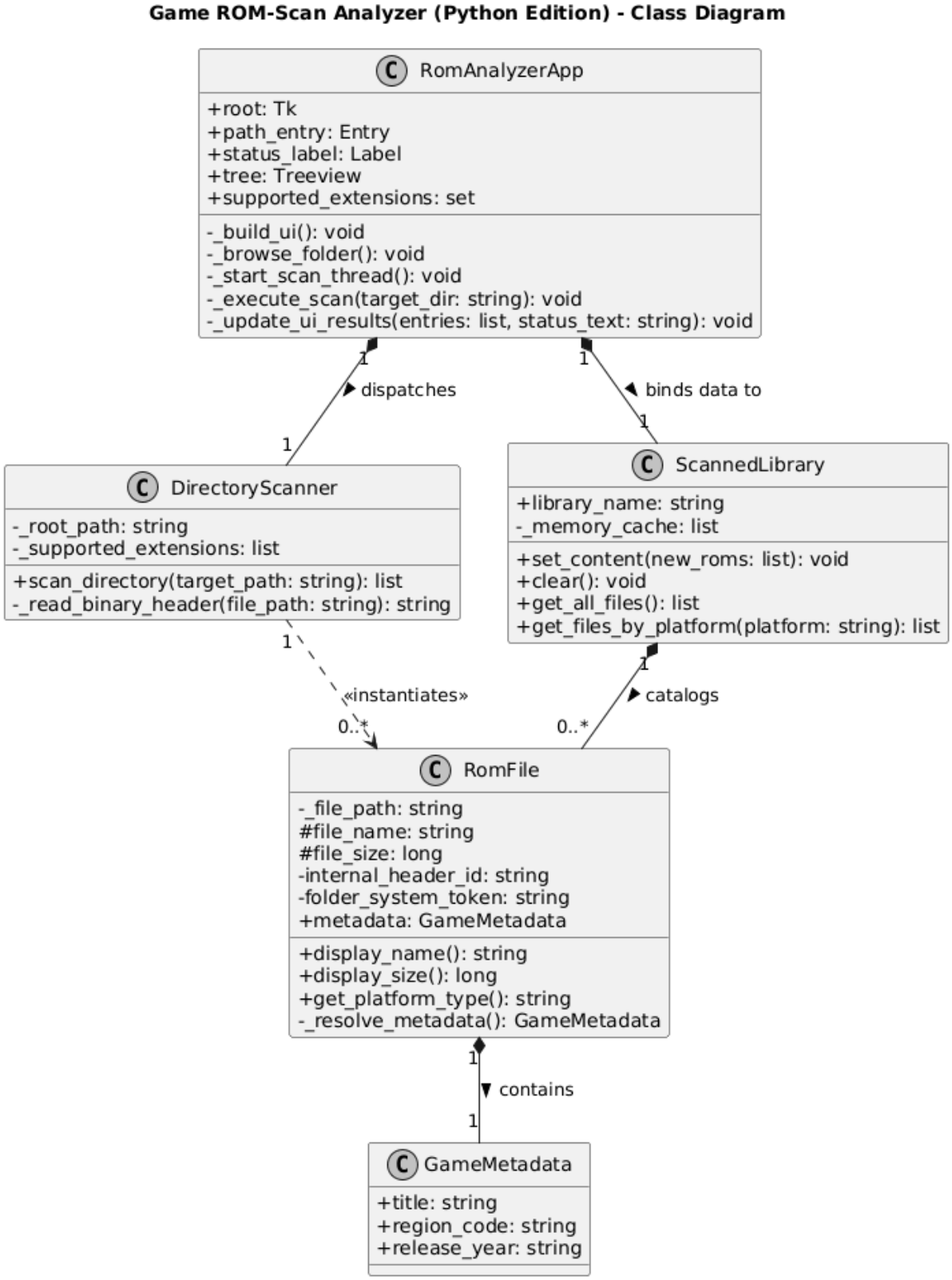
The system maintains explicit operational dependencies with the following software environments:

Operating System Runtime: Runs within native Windows 10/11 x64, macOS 12+, or Linux Kernel 5.4+ architectures.

- **Core File System Interaction APIs:** Relies on Python's built-in os.walk generator loops to stream directory names lazily into memory.

- **Plaintext Configuration File:** Reads an external database configuration asset named systems.txt located within the execution directory root. This file contains text rows that populate the dynamic UI filters

6. System Features/Modules



5.1. Scan Directory for ROMs

5.1.1 Description and Priority

The system parses a user-specified local folder structure to identify valid game files based on internal signature headers. **Priority: High.**

5.1.2 Stimulus/Response Sequences

Stimulus: The User selects a root folder path and triggers the execution of the scanner.

Response: The system references the local File System, reads binary headers, and populates the library display with matches

5.1.3 Functional Requirements

REQ-1.1: The system shall recursively traverse all subdirectories of the target folder path.

REQ-1.2: The system shall parse internal data headers to identify console platforms, matching them regardless of file extension manipulation.

REQ-1.3: The system shall read the file structure path components to check for explicit ES-DE platform directory tokens (fc, snes, gba) as a fallback classification method.

REQ-1.4: The system shall throw a BufferError termination exception if the recursive tracking loop counts more than 10,000 files in the path pool.

5.2 Module: Dynamic Library Filtration & Sorting

5.2.1 Description and Priority

The interface dynamically alters row visibility parameters inside the main presentation treeview grid based on choice criteria.

DOCX

- **Priority: Medium.**

5.2.2 Stimulus/Response Sequences

- **Stimulus: The user changes the selection inside the filter widget entries.**
- **Response: The system filters the collection cache using memory-optimized list comprehensions and updates the visual elements without re-reading files from disk.**

5.2.3 Detailed Functional Requirements

- **REQ-2.1: The system shall load filtering names dynamically at startup from an adjacent systems.txt file.**
- **REQ-2.2: The system shall flush current row visual components whenever a filter update triggers.**
- **REQ-2.3: The system shall re-render corresponding platform rows inside the grid framework within 16 milliseconds.**

6. Nonfunctional Requirements

6.1 Performance

- **NF-1.1: RAM Consumption Boundary: The total processing heap allocation boundary must remain under 512MB RAM at all times when handling collections of up to 10,000 items.**
- **NF-1.2: Deterministic Resource Purging: The backend scanning engine must execute immediate unmanaged handle disposals using Python's with open() syntax and trigger explicit calls to gc.collect() after completing a folder parse.**
- **NF-1.3: Non-Blocking Multi-threading: The filesystem scanning execution loop must run inside a Python threading.Thread worker layout to prevent locking or freezing the main Tkinter window loops.**

6.2 Security

- **NF-2.1: Read-Only Enforcements: The utility must enforce strict read-only parameters ('rb' mode permissions) on all file stream modules to ensure zero modifications are written back to files.**
- **NF-2.2: Memory Extraction Minimization: The scanner must only load the first 16 bytes of a target asset into memory to extract headers, completely avoiding buffering full file arrays.**

6.3 Reliability & Fault Tolerance

- **NF-3.1: Graceful Exception Management: If a directory node or specific file triggers an OS read lock or access clearance exception, the scanner engine must skip that file entity and log the file name error without halting the broader execution thread.**
- **NF-3.2: Configuration Fallbacks: If the dynamic initialization dependency systems.txt file is missing from the directory root, the system must log a warning to the interface console status bar and drop back to using an internal hardcoded backup console array map without crashing.**